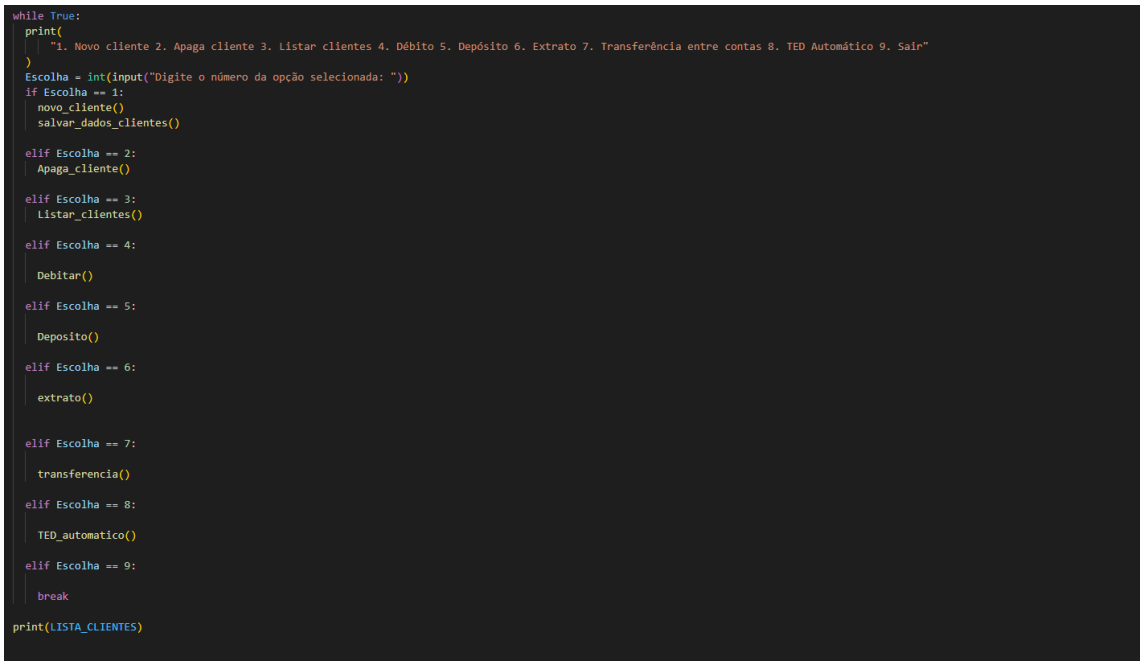


```python
while True:
  print(
      "1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair"
  )
  Escolha = int(input("Digite o número da opção selecionada: "))
  if Escolha == 1:
    novo_cliente()
    salvar_dados_clientes()

  elif Escolha == 2:
    Apaga_cliente()

  elif Escolha == 3:
    Listar_clientes()

  elif Escolha == 4:

    Debitar()

  elif Escolha == 5:

    Deposito()

  elif Escolha == 6:

    extrato()


  elif Escolha == 7:

    transferencia()

  elif Escolha == 8:

    TED_automatico()

  elif Escolha == 9:

    break

print(LISTA_CLIENTES)
```

Inicialmente nosso projeto apresenta um laço while True, onde inicialmente ele irá printar uma mensagem com um menu de opções e seu número correspondente de cada opção.

Após isso é criado uma variável onde dentro dela é atribuído um valor que é digitado pelo usuário através do input, que permite só números inteiros já que as opções são de 1 à 9.

Após isso temos uma condicional “IF” que em sua condição irá chamar a função correspondente ao número 1, caso o usuário atribua o número 1 à variável. Isso também chamará a função salvar dados dos clientes.

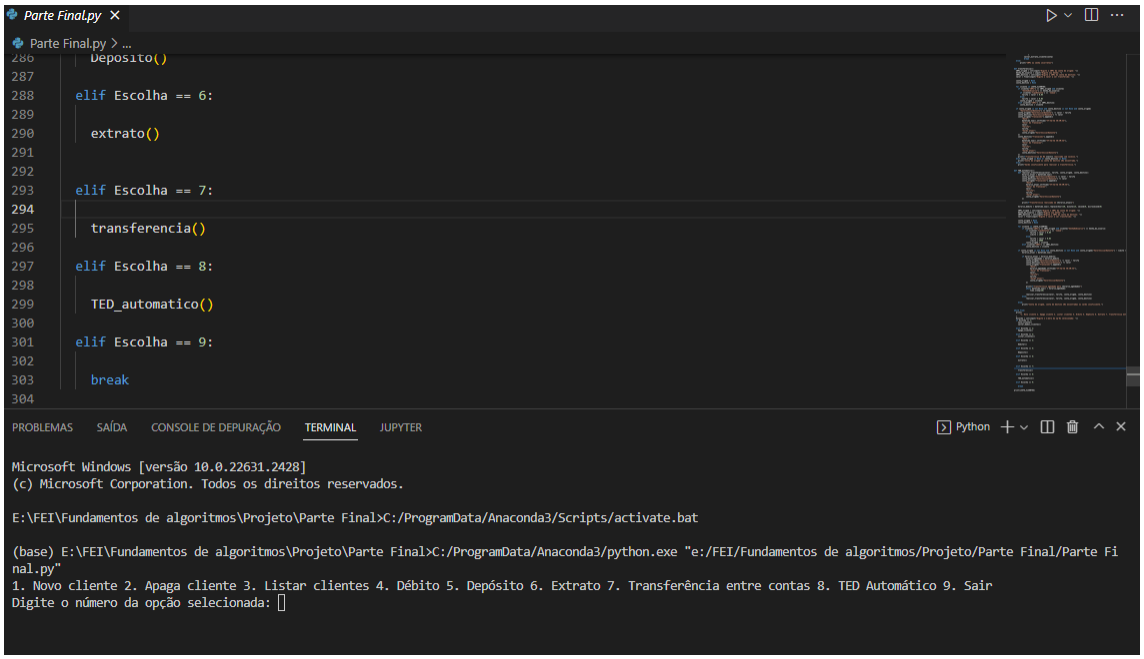
Posteriormente, todas as outras opções estão em Elif, que é quando a condição do If não é satisfeita, mas em seu lugar é satisfeita outra condição. Isso dura até a opção 9, e depois disso o laço é quebrado utilizando o “Break”.

Cada opção é acionada quando um número é preenchido na variável, cada número corresponde a uma função, e se o número for preenchido acionará uma

O menu funciona dentro do loop infinito do while true, que continua sendo executado indefinidamente até que seja solicitado para ele ser quebrado, que é quando a opção 9 é digitada.

Todas as funções que estão dentro desse menu são preenchidas com códigos para seu funcionamento, antes desse menu.

Código em funcionamento:



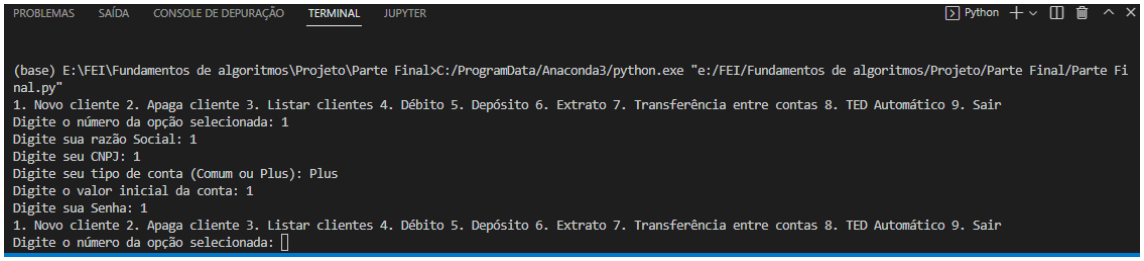
Explicação de cada Função:

- Função 1:

```python
def novo_cliente():
    Razao_social = input("Digite sua razão Social: ")
    CNPJ = int(input("Digite seu CNPJ: "))
    Tipo_de_conta = input("Digite seu tipo de conta (Comum ou Plus): ")
    Valor_inicial_da_conta = float(input("Digite o valor inicial da conta: "))
    Senha_do_usuario = input("Digite sua Senha: ")
    dicionario = {
        "RazaoSocial": Razao_social,
        "CNPJ": CNPJ,
        "TipoDeConta": Tipo_de_conta,
        "ValorInicialDaConta": Valor_inicial_da_conta,
        "SenhaDoUsuario": Senha_do_usuario,
        "Transacoes": []
    }
    LISTA_CLIENTES.append(dicionario)
```

Essa função cria um novo cliente, ela cria variáveis que seus valores são atribuídos com o usuário preenchendo cada opção que aparece, isso é feito usando o input, em alguns é usado o int quando precisa de números inteiros que é o caso do cnpj, e float quando é saldo inicial da conta. Juntamente com isso a função cria um dicionário onde dentro desse dicionário cada chave tem o nome do tipo de informação como cnpj, razão social, e dentro de cada chave é preenchida justamente o valor da variável que tem mesmo nome da chave. Isso é feito pois se precisa ter cada dicionário como informação de cada cliente separadamente, e isso é usado quando precisa-se iterar para pegar cada informação nos códigos posteriores. E por fim cada dicionário criado é adicionado a lista clientes que foi criada como vazia no inicio do código antes das funções, usando o método append.

Funcionamento do código:



Função 2:

```python
def Apaga_cliente():
    CNPJdel = int(input("Digite o CNPJ do cliente a ser removido: "))
    for cliente in LISTA_CLIENTES:
        if cliente['CNPJ'] == CNPJdel:
            LISTA_CLIENTES.remove(cliente)
            print("Cliente removido com sucesso.")
            break
    else:
        print("CNPJ não encontrado.")
```

Essa é a função que apaga cliente, ela cria uma variável que é preenchido seu valor pelo cliente, que é o cnpj da conta que quer ser removida, utiliza o laço de repetição For, que itera em uma variável cliente, cada elemento da lista clientes, sendo cada elemento da lista um dicionário onde estão os dados de cada cliente criado. É checado que se o cliente que o cnpj que foi preenchido for igual a chave cnpj que está atribuída a variável cliente daquele momento, irá ser removido o dicionário que está dentro da lista clientes, esse dicionário é o do cnpj correspondente ao dicionário que foi atribuído a variável cliente daquele momento.

Funcionamento do código:

```
Digite o número da opção selecionada: 1
Digite sua razão Social: 1
Digite seu CNPJ: 1
Digite seu tipo de conta (Comum ou Plus): Plus
Digite o valor inicial da conta: 1
Digite sua Senha: 1
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 2
Digite o CNPJ do cliente a ser removido: 1
Cliente removido com sucesso.
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada:
```

Função 3:

```python
def Listar_clientes():
    print("LISTA DE CLIENTES:")
    for cliente in LISTA_CLIENTES:
        print(cliente)
    print()
```

Essa função simplesmente lista os clientes, ela gera linhas onde cada uma é um dicionário, os dados de cada cliente criado. Ela primeiro printa a mensagem que diz o que vai ser exibido, depois usa o laço de repetição For, iterando sobre a variável cliente os dicionários que estão dentro da lista clientes, e conforme isso vai sendo satisfeito é printado a variável cliente que foi iterada com o dicionário, como é um laço de repetição ela percorrerá toda a lista, e sempre printando os dados dos clientes criados.

Funcionamento do código:

```
Digite seu CNPJ: 2
Digite seu tipo de conta (Comum ou Plus): Plus
Digite o valor inicial da conta: 2
Digite sua Senha: 2
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 3
LISTA DE CLIENTES:
{'RazaoSocial': '1', 'CNPJ': 1, 'TipoDeConta': 'Plus', 'ValorInicialDaConta': 1.0, 'SenhaDoUsuario': '1', 'Transacoes': []}
{'RazaoSocial': '2', 'CNPJ': 2, 'TipoDeConta': 'Plus', 'ValorInicialDaConta': 2.0, 'SenhaDoUsuario': '2', 'Transacoes': []}

1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada:
```

Função 4:

```python
def Debitar():
  CNPJ = int(input("Digite seu CNPJ: "))
  Senha_do_usuario = input("Digite sua Senha: ")
  valor = float(input("Digite o valor a ser debitado: "))
  for conta in LISTA_CLIENTES:
    if conta["CNPJ"] == CNPJ and conta["SenhaDoUsuario"] == Senha_do_usuario:
      if conta['TipoDeConta'] == 'Comum':
        tarifa = valor * 0.05
        limite = 1000
      else:
        tarifa = valor * 0.03
        limite = 5000
      if conta["ValorInicialDaConta"] + limite >= valor + valor*tarifa:
        conta["ValorInicialDaConta"] -= valor - tarifa
        conta["Transacoes"].append({
            "Data":
            datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "Valor da Transacao":
            valor,
            "Tarifa":
            tarifa,
            "Saldo atual":
            conta["ValorInicialDaConta"]
        })
        print(f"Débito de R$ {valor} Realizado")
      else:
        print("Saldo insuficiente para realizar o débito")
      break
  else:
    print("CNPJ não encontrado ou senha inválida")
```

Essa é a função débito, o que ela basicamente faz é retirar dinheiro do saldo inicial da conta do cliente. Ela pede informações de cnpj, senha e valor a ser retirado, esses são atribuídos a variáveis. Usando o laço for que irá iterar sobre uma variável percorrendo cada dicionário da lista, irá com esses dados que foram atribuídos checar se o que foi preenchido existe em algum dicionário dentro da lista, o mesmo com a senha. Caso isso seja verdadeiro, terá outra condição para conferir o tipo de conta, onde é pego a chave tipo de conta dentro da variável conta, que contem os dados do cliente, e essa se for comum cria uma variável tarifa que tem o percentual de 5% e outra variável que tem o limite de 1000, usando o else para quando não for comum a conta, e sim Plus, atribui outros valores as variáveis, sendo tarifa: 3% e limite 5000. Após isso tem outra condição que confere o valor inicial da conta que se a soma dele com o limite (que é o valor que o banco da de crédito para usar, que varia de acordo com o tipo de conta) for maior ou igual ao valor ao qual se quer debitar, somado com a tarifa, o débito será feito, isso confere se tem saldo suficiente na conta para realizar o débito. Também é adicionado os dados da transação a chave “transações” do dicionário daquele cliente que realizou o débito (isso é para registrar as movimentações para utilizar posteriormente no extrato). E por fim printa que o débito foi realizado e seu valor. Mas caso a condição não for satisfeita é printada uma mensagem informando que o saldo é insuficiente para realizar o débito. E se a condição inicial do preenchimento de dados não for satisfeita é printada mensagem que é inválida a senha ou cnpj ao qual foi preenchida.

Funcionamento do código:



Função 5:

```python
def Deposito():
  CNPJ = int(input("Digite seu CNPJ: "))
  deposito = float(input("Digite o valor a ser depositado: "))

  for conta in LISTA_CLIENTES:
    if conta["CNPJ"] == CNPJ:
      conta["ValorInicialDaConta"] += deposito
      conta["Transacoes"].append({
          "Data":
          datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
          "Valor da Transacao":
          deposito,
          "Tarifa":
          0,
          "Saldo atual":
          conta["ValorInicialDaConta"]
      })
      print(f"Depósito de R$ {deposito} Realizado")

    else:
      print("CNPJ inválido")
    break
```

Essa é a função depósito, ela irá apenas colocar um valor dentro do valoriniciladaconta do cliente. Inicialmente é solicitado para preencher uma variável o cnpj para onde o dinheiro vai e o valor do deposito. Depois usando outro laço de repetição é conferido dentro dele se o cnpj preenchido existe, se sim é adicionado o valor do depósito no valorinicialdaconta do cliente daquele cnpj, e depois adicionado os dados do depósito na chave transações do dicionário que corresponde a conta daquele cliente. Se o cnpj preenchido não existir é printado a mensagem dizendo que o mesmo não existe.

Funcionamento do código:

```
Digite sua Senha: 1
Digite o valor a ser debitado: 12345679865
Saldo insuficiente para realizar o débito
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 5
Digite seu CNPJ: 1
Digite o valor a ser depositado: 1
Depósito de R$ 1.0 Realizado
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 5
Digite seu CNPJ: 3
Digite o valor a ser depositado: 4
CNPJ inválido
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada:
```

Função 6:

```python

def extrato():
    CNPJ = int(input("Digite seu CNPJ: "))
    Senha_do_usuario = input("Digite sua Senha: ")

    for conta in LISTA_CLIENTES:
        if conta["CNPJ"] == CNPJ and conta["SenhaDoUsuario"] == Senha_do_usuario:
            print(f"Razão Social: {conta['RazaoSocial']}")
            print(f"CNPJ: {conta['CNPJ']}")
            print(f"Tipo de Conta: {conta['TipoDeConta']}")
            print("Histórico de Operações:")
            for transacao in conta["Transacoes"]:
                print(
                    f"Data: {transacao['Data']} - Valor da Transação: R$ {transacao['Valor da Transacao']} - Tarifa: R$ {tran
                )
            salvar_extrato_cliente(conta)
            break
    else:
        print("CNPJ ou senha incorretos")

```

Essa função imprime as movimentações financeiras do cliente que foi solicitado, ela utiliza a mesma lógica de preenchimento de dados, de laço de repetição para atribuir a uma variável os dados do cliente, e conferir se são os mesmo que foram preenchidos, se tudo está dentro da condição ele imprime as movimentações financeiras do cliente solicitado, mas se a condição não for satisfeita imprime a mensagem que o cnpj e senha são incorretos.

Funcionamento do código:

```
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 6
Digite seu CNPJ: 1
Digite sua Senha: 1
Razão Social: 1
CNPJ: 1
Tipo de Conta: Plus
Histórico de Operações:
Data: 2023-11-16 22:01:48 - Valor da Transação: R$ 12.0 - Tarifa: R$ 0.36 - Saldo Atual: R$ -10.64
Data: 2023-11-16 22:08:23 - Valor da Transação: R$ 1.0 - Tarifa: R$ 0 - Saldo Atual: R$ -9.64
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 6
Digite seu CNPJ: 4
Digite sua Senha: 5
CNPJ ou senha incorretos
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada:
```

Função 7:

```python
def transferencia():
    CNPJ_origem = int(input("Digite o CNPJ da conta de origem: "))
    Senha_do_usuario = input("Digite sua Senha: ")
    CNPJ_destino = int(input("Digite o CNPJ da conta de destino: "))
    valor = float(input("Digite o valor a ser transferido: "))

    conta_origem = None
    conta_destino = None

    for cliente in LISTA_CLIENTES:
        if cliente["CNPJ"] == CNPJ_origem and cliente[
            "SenhaDoUsuario"] == Senha_do_usuario:
            if cliente['TipoDeConta'] == 'Comum':
                tarifa = valor * 0.05
            else:
                tarifa = valor * 0.03
            conta_origem = cliente
        elif cliente["CNPJ"] == CNPJ_destino:
            conta_destino = cliente

    if conta_origem is not None and conta_destino is not None and conta_origem[
        "ValorInicialDaConta"] >= valor:
        conta_origem["ValorInicialDaConta"] -= valor - tarifa
        conta_destino["ValorInicialDaConta"] += valor
        conta_origem["Transacoes"].append({
            "Data":
            datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "Valor da Transacao":
            valor,
            "Tarifa":
            tarifa,
            "Saldo atual":
```

Essa função utiliza os mesmos métodos das anteriores, de preenchimento de informações, laço de repetição para atribuir os dados e conferir se é o mesmo que foi preenchido, com a diferença que se cria duas variáveis já que estará trabalhando com duas contas ao mesmo tempo, e inicialmente se atribui um valor nulo a ela para depois mudar o valor, que se não for preenchido corretamente terá valor nulo, o que será uma condição para a transferência não ser feita. Essa função leva dinheiro de uma conta para outra, e também adicionara a informação ao elemento transações, se as condições não forem satisfeitas será impressa mensagens informando qual foi o erro.

Funcionamento do código:

```
Digite o número da opção selecionada: 1
Digite sua razão Social: 2
Digite seu CNPJ: 2
Digite seu tipo de conta (Comum ou Plus): Plus
Digite o valor inicial da conta: 2
Digite sua Senha: 2
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 5
Digite seu CNPJ: 1
Digite o valor a ser depositado: 12434
Depósito de R$ 12434.0 Realizado
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada: 7
Digite o CNPJ da conta de origem: 1
Digite sua Senha: 1
Digite o CNPJ da conta de destino: 2
Digite o valor a ser transferido: 34
Transferência de R$ 34.0 realizada com sucesso.
1. Novo cliente 2. Apaga cliente 3. Listar clientes 4. Débito 5. Depósito 6. Extrato 7. Transferência entre contas 8. TED Automático 9. Sair
Digite o número da opção selecionada:
```

Função 8:

```python
def TED_automatico():
    def realizar_transferencia(valor, tarifa, conta_origem, conta_destino):
        horario_atual = datetime.now()
        conta_origem["ValorInicialDaConta"] -= valor - tarifa
        conta_destino["ValorInicialDaConta"] += valor
        conta_origem["Transacoes"].append({
            "Data":
            horario_atual.strftime("%Y-%m-%d %H:%M:%S"),
            "Valor da Transacao":
            valor,
            "Tarifa":
            tarifa,
            "Saldo atual":
            conta_origem["ValorInicialDaConta"]
        })

        print(f"Transferência realizada em {horario_atual}")

    horario_debito = datetime.now().replace(hour=20, minute=15, second=0, microsecond=0)

    CNPJ_origem = int(input("Digite o CNPJ da conta de origem: "))
    Senha_do_usuario = input("Digite sua Senha: ")
    CNPJ_destino = int(input("Digite o CNPJ da conta de destino: "))
    valor = float(input("Digite o valor a ser transferido: "))

    conta_origem = None
    conta_destino = None

    for cliente in LISTA_CLIENTES:
        if cliente["CNPJ"] == CNPJ_origem and cliente["SenhaDoUsuario"] == Senha_do_usuario:
            if cliente['TipoDaConta'] == 'Comum':
                elif cliente["CNPJ"] == CNPJ_destino:
                    conta_destino = cliente

    if conta_origem is not None and conta_destino is not None and conta_origem["ValorInicialDaConta"] - limite >= valor
        horario_atual = datetime.now()

        if horario_atual < horario_debito:
            horario_agendado = horario_debito
            conta_origem["ValorInicialDaConta"] -= valor - tarifa
            conta_destino["ValorInicialDaConta"] += valor
            conta_origem["Transacoes"].append({
                "Data":
                horario_agendado.strftime("%Y-%m-%d %H:%M:%S"),
                "Valor da Transacao":
                valor,
                "Tarifa":
                tarifa,
                "Saldo atual":
                conta_origem["ValorInicialDaConta"]
            })

            print(f"Transferência agendada para {horario_agendado}")
            while datetime.now() < horario_agendado:
                time.sleep(30)

            realizar_transferencia(valor, tarifa, conta_origem, conta_destino)
        else:
            realizar_transferencia(valor, tarifa, conta_origem, conta_destino)

    else:
        print("Conta de origem, conta de destino não encontradas ou saldo insuficiente.")
```

Essa função determina um horário para ser realizada uma transferência de dinheiro entre contas, ela já determina nos parênteses condições para isso ser feito, essa são variáveis que serão preenchidas posteriormente, ela utiliza módulos que foram importados, para dessa forma utilizar os horários atuais ela atribui a variável o horário atual utilizando esse módulos, e determina como será transferido os valores de uma conta para a outra, e adiciona as informações da transação no elemento transações essa função está criada mas ela precisa ser chamada para funcionar. É atribuído a variável do horário de agendamento da transferência um valor pré determinado, que também utiliza dos módulos importados. Posteriormente é

solicitado o preenchimento dos dados, e esses são utilizados para validar que a condição é verdadeira para acionar a função e a transferência ser agendada, quando o horário pré estabelecido for verdadeiro a transferência será feita.

Funcionamento do código:

